INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

CORE102

NETWORK AUTOMATION AND INTEGRATED SECURITY CHALLENGE

Laboratory 5 — Practical Laboratory Report

Student information	Details
Student name	Clement Eneji
Registration number	C11_26_OCO_17773
Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Platform	Kali Linux / Ubuntu Linux authorised laboratory environment
Submission date	3 October 2026
Maximum score	100 marks

Assessment evidence report

Contents

01 Part 1 — Automation Plan and Scope

02 Part 2 — Bash Network Inventory

03 Part 3 — Controlled Bash Ping Sweep

04 Part 4 — Python Service Checker

05 Part 5 — Offline PCAP Summary with Scapy

06 Part 6 — Integrated Security Challenge

07 Part 7 — Code Review Checklist

08 Operational Summary

09 Reflection Questions

10 Evidence and Submission Checklist

Authorised network scope: 192.168.56.0/24

Part 1 — Automation Plan and Scope

The laboratory automation is restricted to the authorised private network 192.168.56.0/24. The control plan below defines the permitted input, expected output, operational limit and safe failure behaviour for each tool.

Tool	Permitted input	Output	Timeout / limit	Failure behaviour
Network inventory network_inventory.sh	Local host only	Timestamped text report	Read-only commands	Reports command errors and exits on failure
Ping sweep authorised_ping_sweep.sh	192.168.56.1–192.168.56.254	Responsive hosts (.txt)	1 second per probe (-c 1, -W 1)	Rejects non-authorised prefix; exit 2
Service checker service_check.py	One authorised host in 192.168.56.0/24	Selected TCP port state	1 second per port	Rejects invalid IP / out-of-scope host; exit 1/2/3
PCAP summary pcap_summary.py	Existing local .pcap/.pcapng	Protocol, endpoint and TCP-port counts	Offline read; supplied file only	Rejects missing or unsupported PCAP; exit 1/2

Scope control: Network actions are limited to the explicitly authorised private laboratory range. The ping-sweep script uses a fixed prefix gate, while the service checker uses an IP-network membership test before opening any socket.

Safety control: Network probes are bounded by a one-second timeout and the PCAP exercise reads an existing file offline rather than sniffing live traffic.

Part 2 — Bash Network Inventory

The network inventory script collects local network state without modifying network configuration. It writes a timestamped report and uses strict Bash error handling.

2.1 Strict Bash options

set -e: Stops the script when a command returns a non-zero status, preventing execution from continuing after a command failure.

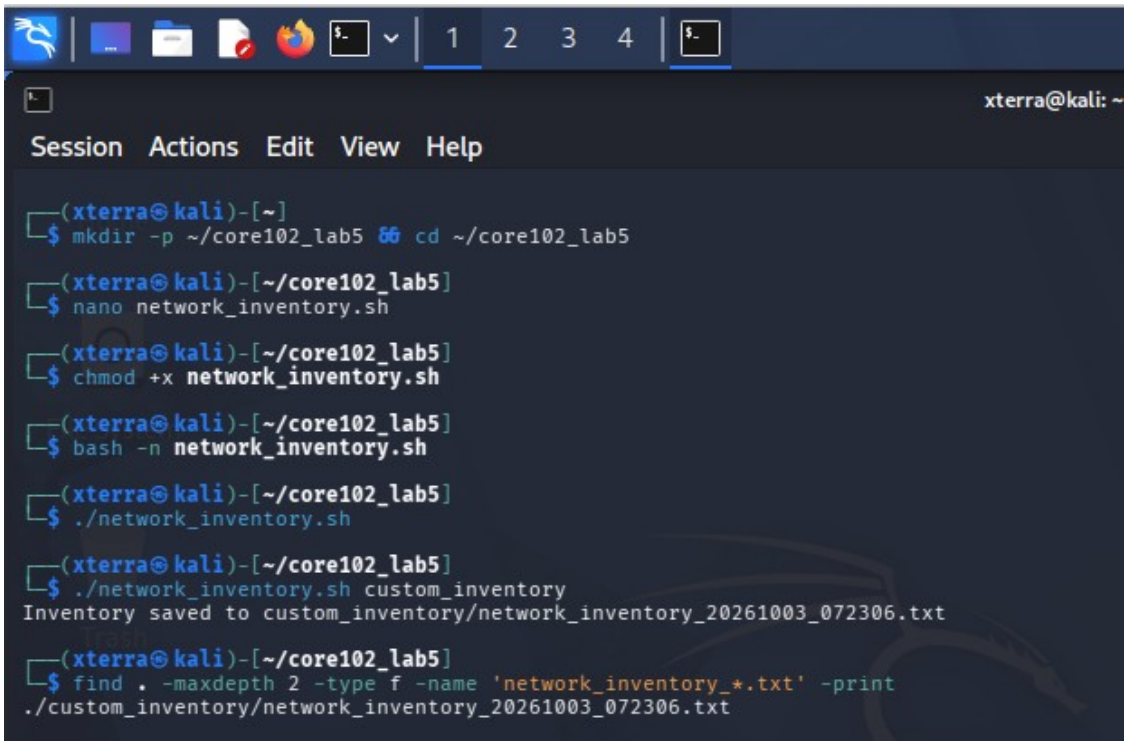
set -u: Treats expansion of an unset variable as an error, reducing failures caused by missing variables or typographical mistakes.

set -o pipefail: Makes a pipeline fail when a command within the pipeline fails, rather than allowing a successful final command to hide an earlier failure.

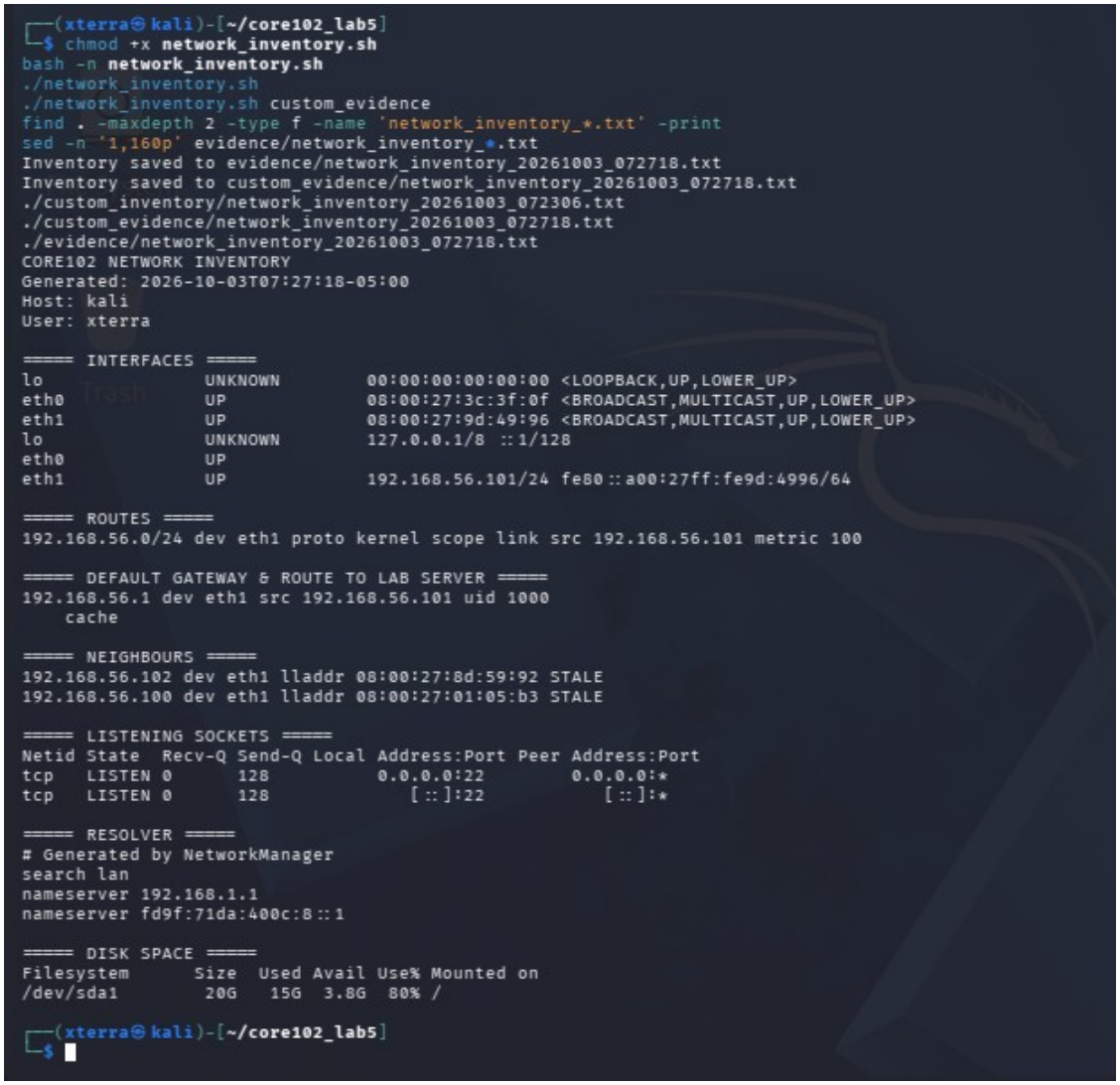
2.2 Default and custom output directories

Default directory: Running ./network_inventory.sh created a timestamped report in the evidence directory.

Custom directory: Running ./network_inventory.sh custom_evidence directed the timestamped report to the specified custom directory. The submitted evidence also shows a custom_inventory run.



Evidence 2.1 — Syntax check and network inventory execution with timestamped output.



Evidence 2.2 — Generated inventory report and recorded network state.

2.3 Default gateway and route enhancement

Enhancement: The inventory was extended to record the default route and the route Linux would use to reach 192.168.56.1. The recorded result identifies interface eth1 and source address 192.168.56.101 for the laboratory route.



Evidence 2.3 — Inventory script showing the added route/gateway collection commands.

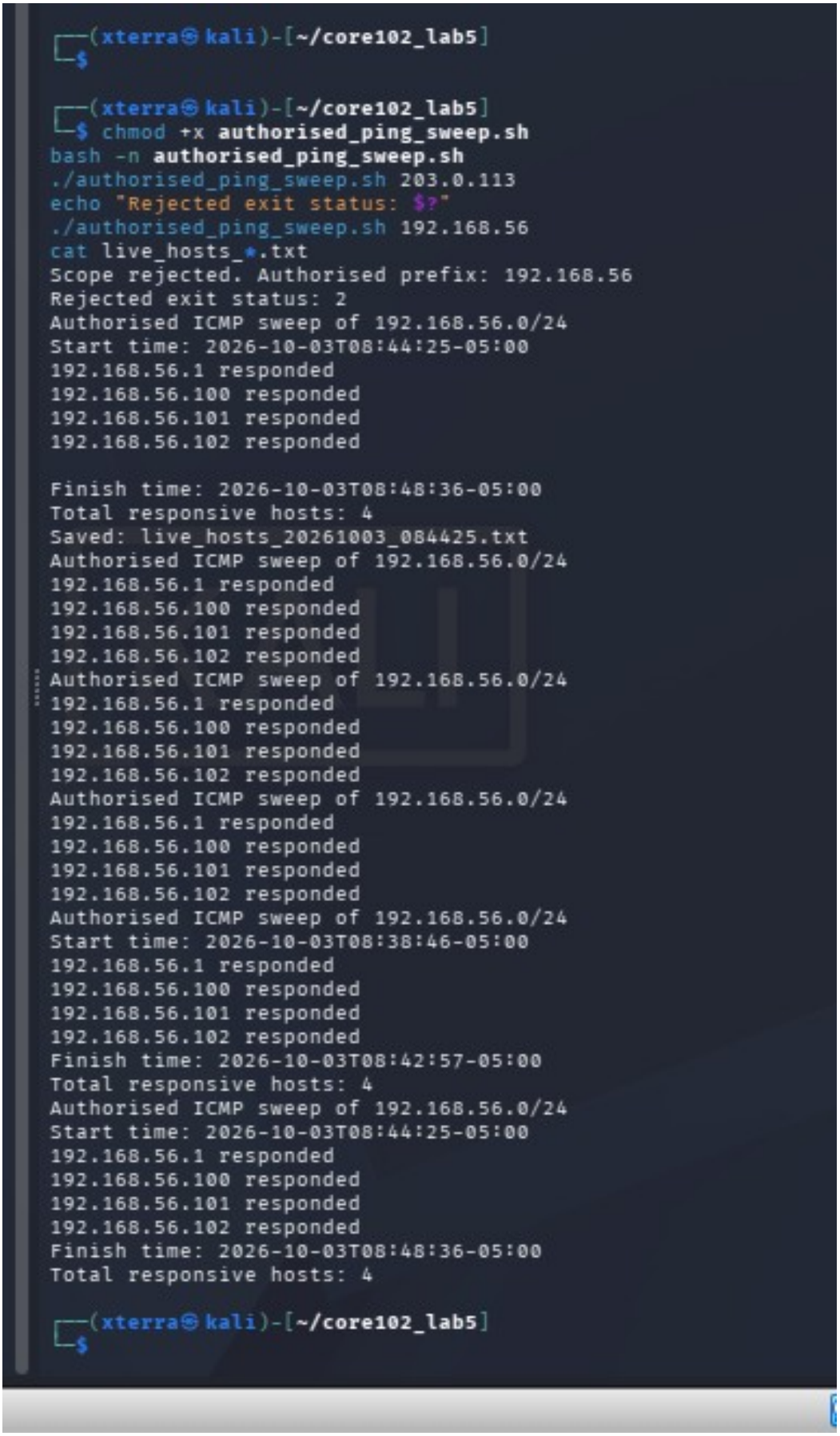
2.4 Read-only behaviour

Result: The inventory commands are status-retrieval operations: ip -br link, ip -br addr, ip route, ip neigh show, ss -tuln, /etc/resolv.conf inspection and df -h. The only intended disk change is creation of the report file in the selected output directory.

Part 3 — Controlled Bash Ping Sweep

The sweep enforces the approved 192.168.56 prefix before entering the host loop. Each ICMP probe is limited to one packet with a one-second wait.

Out-of-scope validation: The test using 203.0.113 was rejected before any sweep was performed, returning exit status 2 and displaying the authorised-prefix message.



Evidence 3.1 — Rejected out-of-scope test followed by the authorised sweep and responsive-host results.

Authorised sweep result: The successful sweep recorded four responsive hosts: 192.168.56.4, 192.168.56.100, 192.168.56.101 and 192.168.56.102. The evidence also records the sweep start/finish information and a total responsive-host count of four.

Why an online host may not answer ICMP: A host can be operational while ignoring ICMP echo requests because of a host firewall, network filtering, disabled ICMP handling, routing restrictions or another security policy. Therefore, lack of an ICMP response is not proof that the host is offline.

3.1 Scope enforcement implementation



Evidence 3.2 — Authorised ping-sweep script showing the fixed prefix gate and one-second ICMP probe.

Part 4 — Python Service Checker

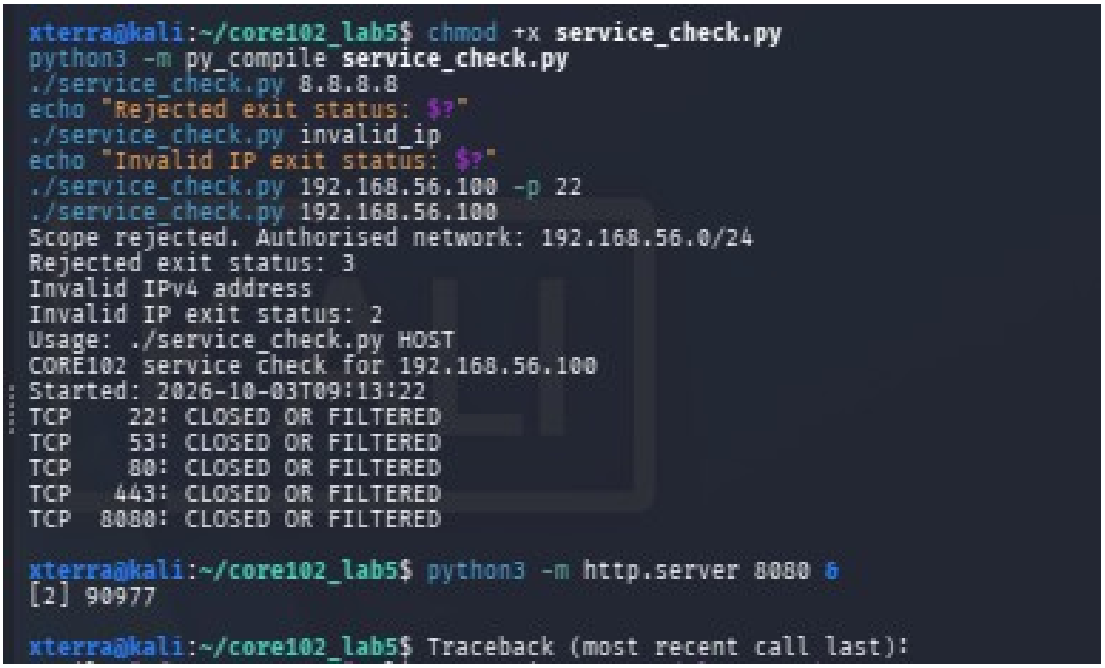
The service checker validates the target as an IPv4 address, confirms membership of 192.168.56.0/24, and uses socket.create_connection() with a one-second timeout for the approved TCP ports 22, 53, 80, 443 and 8080.

4.1 Input and scope validation

Invalid input: A non-IPv4 value is rejected with an invalid-address message and exit status 2.

Out-of-scope input: The address 8.8.8.8 is rejected by the authorised-network check and the recorded exit status is 3.

Authorised target: The checker was run against 192.168.56.100/101 in the supplied evidence and returned bounded states for the selected ports.

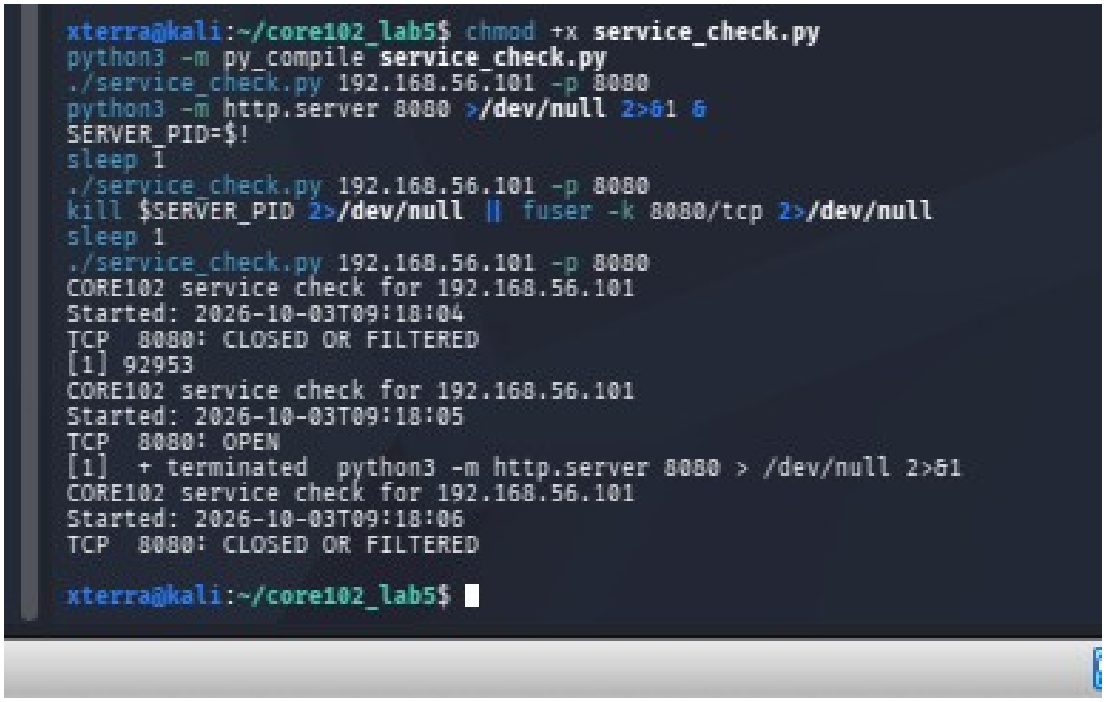


Evidence 4.1 — Invalid input, out-of-scope address rejection and authorised service-check execution.

4.2 Service availability change detection

Test service: A temporary Python HTTP server was started on TCP port 8080. The service checker then reported TCP 8080 as OPEN. After the temporary server process was terminated, a subsequent check reported TCP 8080 as CLOSED OR FILTERED.

Interpretation: The change demonstrates that the checker can detect a change in TCP service reachability. The result does not distinguish a closed service from a filtered service when the connection attempt does not succeed.



Evidence 4.2 — TCP 8080 changes from OPEN while the HTTP test service is running to CLOSED OR FILTERED after termination.

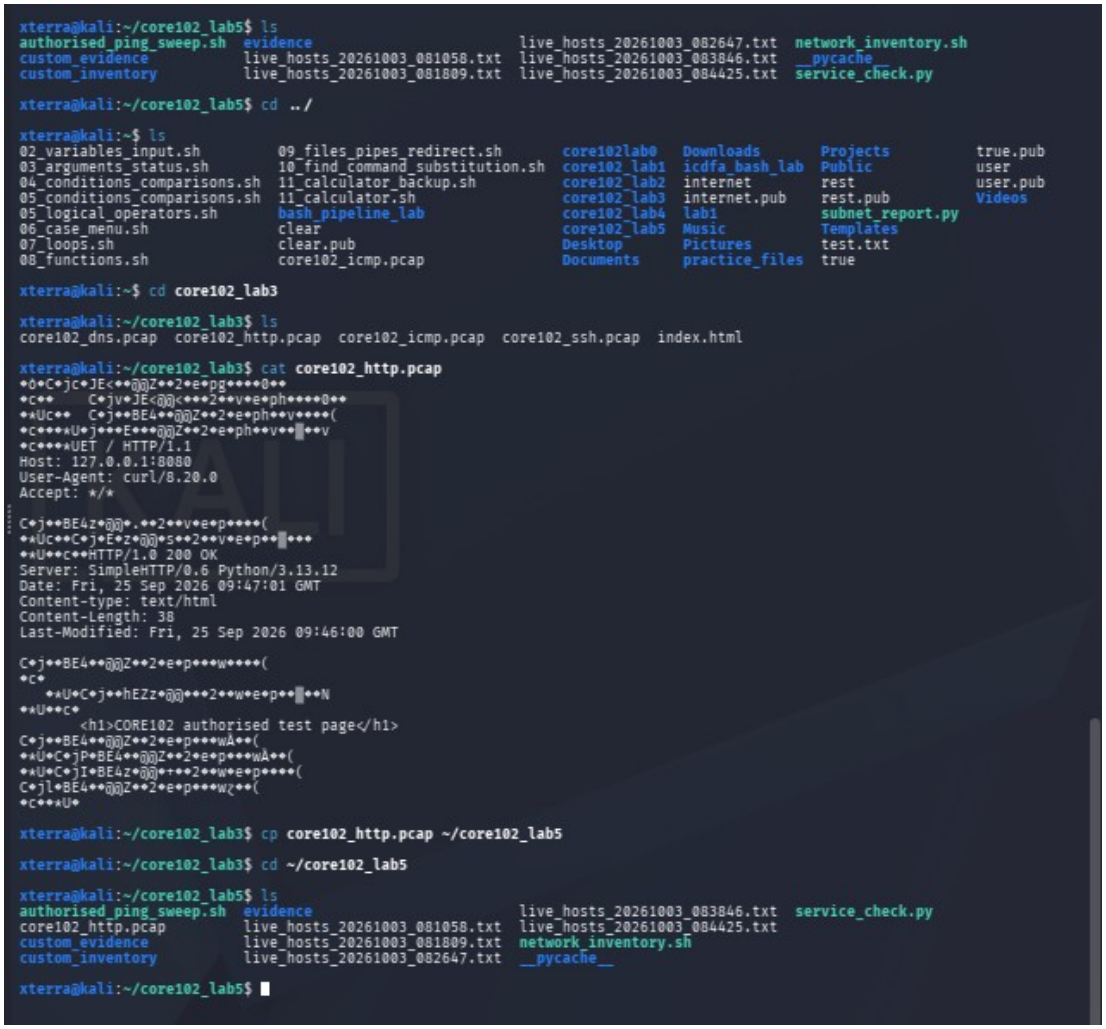
4.3 Bounded socket checks

Timeout control: Each socket connection uses timeout=1, ensuring the checker does not wait indefinitely on a non-responsive port.

Part 5 — Offline PCAP Summary with Scapy

The supplied evidence uses core102_http.pcap copied from the earlier laboratory and analyses it offline. No live packet sniffing is required for the summary operation.

PCAP validation: The program checks that the supplied path is a file and that its extension is .pcap or .pcapng before opening it with PcapReader.



Evidence 5.1 — Authorised PCAP present in the laboratory workspace and copied into the Lab 5 directory.

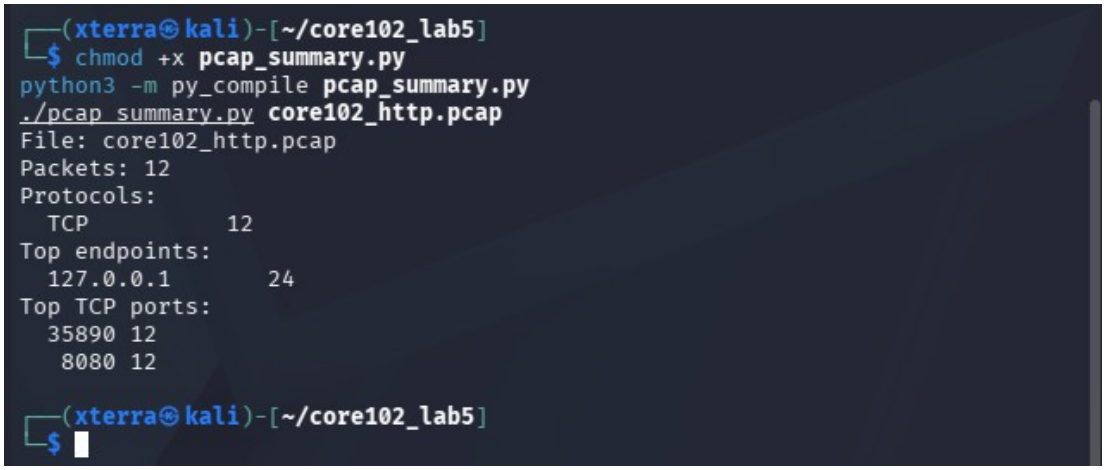
5.1 Parsing and protocol results

Packet total: The supplied execution reports 12 packets.

Protocol result: All 12 packets were classified as TCP.

Endpoint result: 127.0.0.1 appears as the top endpoint with 24 endpoint occurrences, reflecting source and destination accounting across the packets.

TCP ports: Ports 35890 and 8080 each occur 12 times in the TCP port counter.



Evidence 5.2 — pcap_summary.py compilation and offline summary output for core102_http.pcap.

5.2 Required analysis questions

1. Packet-total comparison: The script reports 12 packets. No separate capinfos or Wireshark Statistics result is included in the submitted evidence, so an independent second-source packet-total comparison is not claimed.

2. Why both source and destination ports are counted: Every TCP segment has both a source and destination port. Counting both captures the communicating endpoints of each segment and allows the ephemeral client port and service port to be identified.

3. DNS counter enhancement: DNS traffic can be counted by checking UDP or TCP port 53 before the general protocol classification. A suitable enhancement is shown below.

```
if TCP in packet:
    tcp_ports.update([packet[TCP].sport, packet[TCP].dport])
    if packet[TCP].sport == 53 or packet[TCP].dport == 53:
        protocols['DNS'] += 1
    protocols['TCP'] += 1
elif UDP in packet:
    if packet[UDP].sport == 53 or packet[UDP].dport == 53:
        protocols['DNS'] += 1
    protocols['UDP'] += 1
```

4. Port-based classification limitation: Port numbers are only an indicator. Applications can use non-standard ports, ports can be reused by different services, and encryption can prevent application-layer identification from port information alone.

Part 6 — Integrated Security Challenge

The integrated evidence was reviewed against the required final-state controls. The table below distinguishes what the submitted evidence demonstrates from controls that remain unresolved.

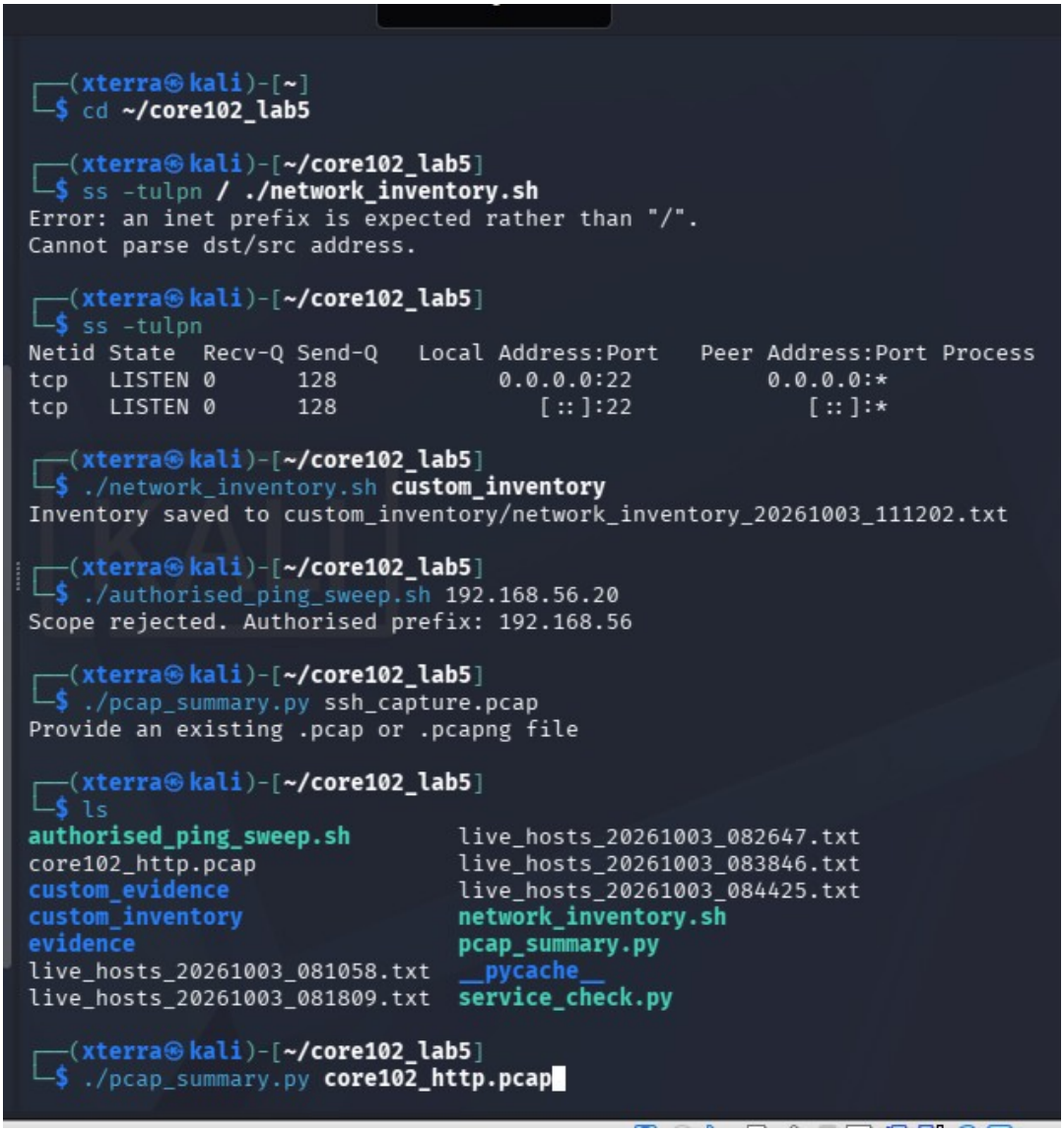
Control	Evidence result	Conclusion
Address and route	Recorded result: both VMs were assigned within 192.168.56.0/24 and the laboratory host route was valid.	Private laboratory topology was documented and scoped.
Firewall policy	UFW was active with default deny incoming and allow outgoing. However, the captured rules show 22/tcp ALLOW IN Anywhere and 22/tcp (v6) ALLOW IN Anywhere (v6).	The default incoming policy is restrictive, but the required SSH source restriction to 192.168.56.0/24 is not demonstrated by the captured rule set.
SSH key authentication	An SSH session to 192.168.56.20 was established. The screenshot also shows: 'Identity file .../.ssh/id_rsa not accessible: No such file or directory'.	SSH connectivity is demonstrated, but key-based authentication is not conclusively verified by the supplied evidence.
Listening services	ss -tulpn shows TCP listeners on 0.0.0.0:22 and [::]:22.	The captured socket evidence supports SSH as the listening service and shows no additional listening TCP service in that output.
Host discovery	The authorised sweep returned four responsive hosts: .4, .100, .101 and .102 within 192.168.56.0/24.	Controlled host discovery was demonstrated inside the authorised private range.
PCAP summary	core102_http.pcap produced 12 TCP packets, with loopback endpoint 127.0.0.1 and ports 35890/8080. The attempted ssh_capture.pcap was not present in the captured workspace.	Offline PCAP parsing was demonstrated, but the submitted evidence does not establish the required SSH packet capture.
Cleanup	The http.server process was successfully stopped with pkill -f http.server. Subsequent commands intended as file-cleanup commands returned pkill usage errors.	The test HTTP service was stopped; complete temporary-file cleanup is not demonstrated by the captured evidence.

6.1 Firewall and SSH evidence

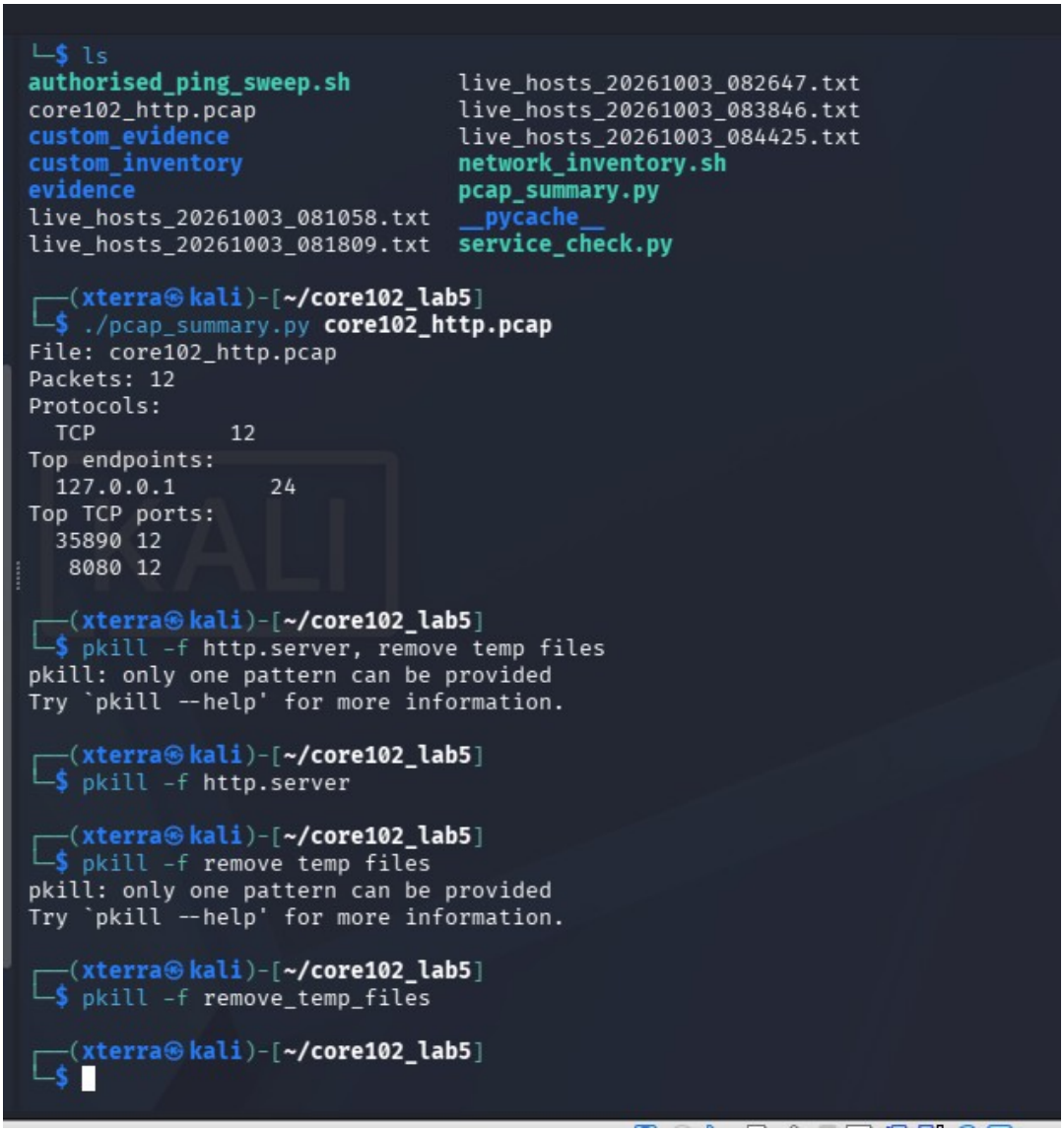


Evidence 6.1 — UFW status and SSH connection attempt/session. The captured UFW rule set is recorded exactly as observed.

6.2 Listening services and PCAP evidence



Evidence 6.2 — Listening sockets, inventory execution, scope rejection and the missing ssh_capture.pcap test.



Evidence 6.3 — Successful offline analysis of core102_http.pcap and cleanup commands.

6.3 Final network state

Verified controls: The authorised private subnet, controlled ICMP discovery, bounded TCP service checks, local network inventory, offline PCAP parsing and SSH listening service were evidenced.

Unresolved issues: The captured UFW rule permits SSH from Anywhere rather than only 192.168.56.0/24; the SSH key file was reported missing during the connection command; and the required ssh_capture.pcap was not available in the captured workspace.

Cleanup state: The temporary HTTP server was stopped. The evidence does not establish removal of all temporary files, so the retained Lab 5 evidence files should be treated as deliberate submission artefacts rather than proof of complete filesystem cleanup.

Part 7 — Code Review Checklist

Script	Interpreter	Scope bounding	Syntax verification
network_inventory.sh	#!/usr/bin/env bash	Local host / interface state; no remote target input	bash -n network_inventory.sh
authorised_ping_sweep.sh	#!/usr/bin/env bash	Fixed 192.168.56 prefix check	bash -n authorised_ping_sweep.sh
service_check.py	#!/usr/bin/env python3	ipaddress membership check for 192.168.56.0/24	python3 -m py_compile service_check.py
pcap_summary.py	#!/usr/bin/env python3	Offline file and extension validation; Scapy packet parsing	python3 -m py_compile pcap_summary.py

Code review findings

- Each submitted script uses an appropriate interpreter line.
- Inputs are validated before the corresponding network action or file parse begins.
- The authorised address scope is explicit in the ping sweep and service checker.
- Network operations use bounded packet counts or connection timeouts.
- Outputs include timestamps for the network inventory and service checker; the PCAP summary identifies its input file.
- Error messages explain rejected scope or invalid input.
- No passwords, private keys or tokens are included in the supplied scripts or report text.

Operational Summary

The Lab 5 exercise produced a repeatable set of network-automation checks for the authorised 192.168.56.0/24 laboratory environment. The Bash inventory generated timestamped local network evidence and was extended to record the route toward the laboratory address. The controlled ICMP sweep rejected an out-of-scope prefix before probing and identified four responsive hosts inside the authorised range.

The Python service checker validated IPv4 input and network scope and used one-second socket timeouts. The temporary HTTP service on TCP 8080 was observed as OPEN while running and CLOSED OR FILTERED after termination, demonstrating service-change detection. The Scapy utility successfully parsed the supplied core102_http.pcap offline and reported 12 TCP packets, loopback endpoint activity and ports 35890/8080.

The integrated evidence also identified controls requiring correction before the environment can be described as fully compliant with the challenge requirements. The captured UFW rules allow SSH from Anywhere rather than only 192.168.56.0/24. The SSH command reported that the specified private-key file was not accessible, so key-based authentication is not independently established by the evidence. In addition, the requested SSH PCAP was not present; the available PCAP evidence is the earlier HTTP test capture. These points are recorded as unresolved rather than being represented as verified controls.

The final evidence therefore separates observed results from conclusions and avoids treating an unsuccessful or incomplete test as proof of a successful security control.

Reflection Questions

1. Which validation control most effectively prevented an out-of-scope action? The fixed authorised-prefix gate in authorised_ping_sweep.sh is the clearest preventive control because it checks the supplied prefix before the host loop begins. A non-authorised prefix returns exit status 2 and no ICMP sweep is started.

2. What can a service checker conclude, and what can it not conclude, from a timeout? It can conclude that a TCP connection was not successfully established within the configured one-second timeout. It cannot by itself conclude that the service is definitely closed: packet filtering, host firewall rules, routing problems, congestion or temporary unavailability can also produce a timeout. The script therefore reports the state as CLOSED OR FILTERED.

3. How did automation improve repeatability compared with running commands manually? The scripts standardised the scope, validation sequence, timeout values, output format and evidence generation. The same commands can be repeated with less variation, while timestamps and saved reports provide a consistent record for comparison.

4. Which additional control would you add before these scripts could support a larger authorised environment? I would add a centrally maintained allowlist of approved CIDRs/hosts with explicit change control and audit logging. The scripts should validate every target against that allowlist, record the operator, target, timestamp and result, and retain the evidence for review. This would reduce the risk of accidental expansion of the testing scope as the environment grows.

Evidence and Submission Checklist

Assessment item	Evidence / answer	Status
Student identity	Clement Eneji and C11_26_OCO_17773 included	Complete
Automation plan	Scope, inputs, outputs, limits and failure behaviour documented	Complete
Network inventory	Syntax, execution, timestamped report and route enhancement evidenced	Complete
Ping sweep	Out-of-scope rejection, authorised results and ICMP limitation explained	Complete
Service checker	Invalid/out-of-scope validation and 8080 change detection evidenced	Complete
PCAP summary	Offline parse, 12-packet result and protocol/endpoint/port interpretation evidenced	Complete
Integrated challenge	Addressing, firewall, SSH, services, discovery and PCAP evidence reviewed	Partial — unresolved controls explicitly identified
Code review	Interpreter, scope, bounded operations and syntax checks documented	Complete
Operational summary	Findings, limitations and cleanup state documented	Complete
Reflection	All four required questions answered	Complete

End of report